

TD1 — Modèle de programmation Pthread

Parallélisme à base de thread ()

Auteur : Yahdhih Abdel Wedoud

Date : 18 janvier 2026

Résumé. Ce rapport répond à l'ensemble des questions **Q1 à Q14**. Les réponses sont structurées par sections et les extraits de code sont fournis dans des blocs dédiés, coupables sur plusieurs pages.

Table des matières

1	Contexte et objectifs	1
2	Découverte de l'API Pthread	2
2.1	Hello World!	2
3	Multithread-unsafe : données partagées sans protection	3
3.1	Analyse	3
4	Recherche parallèle du maximum d'un tableau	4
4.1	Cadre	4
5	Les conditions (mutex + variable conditionnelle)	4
5.1	Contexte	4
6	Recherche parallèle du max avec les sémaphores	5
7	Barrière multithread	6
8	Conclusion	7

1 Contexte et objectifs

Le but de ce TD est de se familiariser avec l'API `pthread` afin de comprendre les principes essentiels du parallélisme à base de threads (création, terminaison, partage de mémoire, synchronisation), puis d'implémenter des mécanismes classiques de synchronisation :

- exclusion mutuelle (`mutex`);
- variables conditionnelles;
- sémaphores;
- barrière.

Commande de compilation (générique)

```
1 gcc -Wall -Wextra -O2 -pthread fichier.c -o programme
```

2 Découverte de l'API Pthread

2.1 Hello World!

Objectif. Lancer un nombre de threads fourni en argument ; chaque thread affiche une chaîne. La terminaison est attendue via `pthread_join`. L'ordre d'affichage est non déterministe (ordonnancement).

Code — Q1

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 static void *hello_world(void *arg)
6 {
7     (void)arg;
8     printf("Hello world!\n");
9     return NULL;
10 }
11
12 int main(int argc, char **argv)
13 {
14     if (argc < 2) {
15         fprintf(stderr, "Usage: %s <nthreads>\n", argv[0]);
16         return 1;
17     }
18
19     int nthreads = atoi(argv[1]);
20     pthread_t tids[nthreads];
21
22     for (int i = 0; i < nthreads; i++)
23         pthread_create(&tids[i], NULL, hello_world, NULL);
24
25     for (int i = 0; i < nthreads; i++)
26         pthread_join(tids[i], NULL);
27
28     return 0;
29 }
```

Objectif. Chaque thread affiche un rang ; la chaîne est construite avant la création. **Principe.** Allouer une structure par thread contenant une chaîne préconstruite, puis libérer la mémoire dans le thread.

Code — Q2 (extrait minimal)

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
```

```
4 #include <string.h>
5
6 typedef struct { char *msg; } payload_t;
7
8 static void *run(void *arg)
9 {
10     payload_t *p = (payload_t*)arg;
11     fputs(p->msg, stdout);
12     free(p->msg);
13     free(p);
14     return NULL;
15 }
```

Objectif. Passer un préfixe et un identifiant, le thread gère l’affichage. **Note.** Éviter les casts non portables (int)arg. Utiliser intptr_t si nécessaire.

Code — Q3 (extrait minimal)

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 typedef struct {
6     const char *prefix;
7     int rank;
8 } args_t;
9
10 static void *run(void *arg)
11 {
12     args_t *a = (args_t*)arg;
13     printf("%s %d\n", a->prefix, a->rank);
14     free(a);
15     return NULL;
16 }
```

3 Multithread-unsafe : données partagées sans protection

3.1 Analyse

Sans synchronisation, l’accès concurrent à des variables partagées peut provoquer des **conditions de course** et rendre le résultat non déterministe.

Observation. La valeur finale varie selon les exécutions et est souvent inférieure au résultat attendu. **Explication.** count++ n’est pas atomique : lecture puis écriture. Deux threads peuvent perdre des incréments par écrasement.

Signification. Empêche certaines optimisations (valeur gardée en registre, suppression d’accès mémoire). **Limite.** Ne garantit ni atomicité, ni exclusion mutuelle, ni synchronisation inter-threads. Donc **insuffisant**.

Solution attendue. Protéger la section critique via un `mutex`. Alternative : opérations atomiques (C11 `stdatomic.h`) si autorisé.

Code — Q6 (section critique)

```
1 pthread_mutex_lock(&mutex);
2 count++;
3 pthread_mutex_unlock(&mutex);
```

4 Recherche parallèle du maximum d'un tableau

4.1 Cadre

On cherche le maximum d'un tableau de `nelt` entiers en utilisant `nthreads` threads. Le tableau est partitionné en blocs contigus (dernier bloc = reste).

Principe.

1. chaque thread calcule un maximum local;
2. le principal récupère les maxima (par exemple `local_max[i]`);
3. le principal calcule le maximum global.

Principe.

1. chaque thread calcule un max local;
2. compare avec `maxv_mt`;
3. met à jour `maxv_mt` sous `mutex`.

5 Les conditions (mutex + variable conditionnelle)

5.1 Contexte

Deux threads *A* et *B*. *B* produit une valeur partagée, *A* doit afficher en garantissant que la valeur de *B* est disponible et cohérente.

Comportements possibles. Blocage, non-déterminisme, lecture incohérente.

Cause. Signal perdu (*lost wakeup*) et absence de prédicat. Une condition doit être attendue en `while` sur un état partagé, sous `mutex`, pour gérer les réveils spurieux et garantir la cohérence.

Solution. Flag `ready` protégé par `mutex` : *A* attend `while (!ready)`; *B* écrit, pose le flag, puis `signal`.

Code — Q10 (mutex + condition + prédicat)

```
1 int valB = 0;
2 int ready = 0;
3
4 void *runA(void *arg)
5 {
```

```

6   int valA = work(0);
7
8   pthread_mutex_lock(&mutex);
9   while (!ready)
10      pthread_cond_wait(&cond, &mutex);
11   int localB = valB;
12   pthread_mutex_unlock(&mutex);
13
14   printf("*A* Valeur de A: %d Valeur de B: %d\n", valA, localB);
15   return NULL;
16 }
17
18 void *runB(void *arg)
19 {
20     int tmp = work(0);
21
22     pthread_mutex_lock(&mutex);
23     valB = tmp;
24     ready = 1;
25     pthread_cond_signal(&cond);
26     pthread_mutex_unlock(&mutex);
27
28     printf("*B* Valeur calculee %d\n", tmp);
29     return NULL;
30 }

```

Idée. Sémaphore initialisé à 0 : *A* fait `sem_wait`, *B* fait `sem_post` après production.

Code — Q11 (sémaphore de synchronisation)

```

1  #include <semaphore.h>
2
3  sem_t doneB; // init à 0
4
5  // A:
6  sem_wait(&doneB);
7  printf("... valB ...");
8
9  // B:
10 valB = work(0);
11 sem_post(&doneB);

```

Objectif. Empêcher l'écrasement : *B* ne produit pas tant que *A* n'a pas consommé, et *A* n'affiche pas tant que *B* n'a pas produit.

Modèle. Buffer de taille 1 avec état `ready` (0 vide / 1 plein) sous mutex + cond, attentes en `while`.

6 Recherche parallèle du max avec les sémaphores

Objectif. Remplacer l'exclusion mutuelle sur `maxv_mt` par un **sémaphore binaire** (init à 1).

Stratégie. Calculer un max local par thread, puis protéger la mise à jour globale avec `sem_wait/sem_post`.

Intérêt. Contention faible : une seule mise à jour globale par thread, en $\mathcal{O}(1)$.

Code — Q13 (mise à jour protégée par sémaphore)

```

1 #include <semaphore.h>
2
3 static sem_t sem_max;
4 static int maxv_mt;
5
6 static void update_global_max(int local_max)
7 {
8     sem_wait(&sem_max);
9     if (local_max > maxv_mt)
10         maxv_mt = local_max;
11     sem_post(&sem_max);
12 }
```

7 Barrière multithread

Objectif. Implémenter une barrière équivalente à `pthread_barrier_t` via mutex + cond.

Point clé. Pour une barrière réutilisable (dans une boucle), ajouter `generation`.

Règle. Le dernier arrivé réarme (`count=0`), incrémente `generation` et réveille tous les threads (broadcast). Les autres attendent en `while` que `generation` change.

Code — Q14 (barrière réutilisable)

```

1 typedef struct {
2     pthread_mutex_t mutex;
3     pthread_cond_t cond;
4     int num;           // threads attendus
5     int count;         // arrivés
6     int generation;    // itération
7 } my_barrier_t;
8
9 int my_barrier_init(my_barrier_t *b, int n)
10 {
11     if (!b || n <= 0) return -1;
12     b->num = n;
13     b->count = 0;
14     b->generation = 0;
15     if (pthread_mutex_init(&b->mutex, NULL) != 0) return -1;
16     if (pthread_cond_init(&b->cond, NULL) != 0) return -1;
17     return 0;
18 }
19
20 int my_barrier_wait(my_barrier_t *b)
21 {
22     if (!b) return -1;
23
24     pthread_mutex_lock(&b->mutex);
25
26     int my_gen = b->generation;
27     b->count++;
28
29     if (b->count == b->num) {
```

```
30     b->count = 0;
31     b->generation++;
32     pthread_cond_broadcast(&b->cond);
33     pthread_mutex_unlock(&b->mutex);
34     return 1; // optionnel : leader
35 }
36
37 while (my_gen == b->generation)
38     pthread_cond_wait(&b->cond, &b->mutex);
39
40 pthread_mutex_unlock(&b->mutex);
41 return 0;
42 }
```

8 Conclusion

Ce TD illustre les aspects fondamentaux de la programmation concurrente avec `pthread` : création/attente de threads, non-déterminisme, dangers des accès concurrents, synchronisations (`mutex`, `cond`, sémaphores) et construction d'une barrière réutilisable.